

Qwen3.6-27B MTP

Guide d'installation et d'utilisation sur **Mac**
avec llama.cpp (serveur local)

Modèle	Format	Plateforme	Accélération
Qwen3.6-27B	GGUF UD-Q4_K_XL	Mac Apple Silicon	Metal + MTP

1. Qu'est-ce que le MTP ?

MTP (Multi-Token Prediction) est une technique de décodage spéculatif intégrée dans les poids du modèle Qwen3.6-27B par Alibaba. Au lieu de prédire un token à la fois, le modèle prédit plusieurs tokens simultanément, offrant une vitesse de génération 1.5 à 2x plus rapide sans perte de qualité.

Unslloth a converti ce modèle en GGUF en préservant les têtes MTP, permettant de l'utiliser localement sur Mac.

⚠ Le MTP nécessite plus de RAM que le mode standard. Les têtes spéculatives sont chargées en parallèle du modèle principal (~18 GB). Sur un Mac avec moins de 32 GB de RAM disponible, il est recommandé de désactiver le MTP.

2. Prérequis

Configuration minimale recommandée :

Élément	Minimum	Recommandé
Mac	Apple Silicon (M1/M2/M3)	M2 Pro / M3 Pro ou mieux
RAM	24 GB (sans MTP)	32 GB ou plus (avec MTP)
Espace disque	25 GB libres	40 GB libres
Connexion	Haut débit	Fibre (~18 GB à télécharger)
macOS	Ventura 13+	Sequoia 15+

3. Installation

Étape 1 — Outils de compilation

Ouvre le Terminal et installe les outils nécessaires :

```
xcode-select --install  
brew install cmake curl
```

Si Homebrew n'est pas installé : `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`

Étape 2 — Cloner la branche MTP de llama.cpp

Le MTP n'est pas encore dans la version officielle. Il faut compiler depuis une branche spéciale :

```
git clone -b mtp-clean https://github.com/am17an/llama.cpp.git
```

Étape 3 — Configurer la compilation

Le flag `-DGGML_CUDA=OFF` est essentiel sur Mac. Metal (GPU Apple Silicon) sera détecté automatiquement :

```
cmake llama.cpp -B llama.cpp/build -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=OFF
```

Étape 4 — Compiler (10-15 minutes)

```
cmake --build llama.cpp/build --config Release -j --clean-first --target llama-  
cli llama-server
```

Le flag `-j` utilise tous les coeurs CPU pour accélérer la compilation.

Étape 5 — Copier les exécutables

```
cp llama.cpp/build/bin/llama-* llama.cpp
```

4. Lancer le serveur

Avec MTP (recommandé si 32 GB+ de RAM disponible)

La première exécution télécharge automatiquement le modèle (~18 GB). Les fois suivantes, le modèle est déjà sur le disque et le démarrage est immédiat.

```
export LLAMA_CACHE="unsloth/Qwen3.6-27B-MTP-GGUF"  
./llama.cpp/llama-server \  
-hf unsloth/Qwen3.6-27B-MTP-GGUF:UD-Q4_K_XL \  
-ngl 99 \  
-c 8192 \  
-fa on \  
-np 1 \  
--spec-type mtp \  
--spec-draft-n-max 3
```

Description des flags

Flag	Valeur	Description
-hf	unsloth/...:UD-Q4_K_XL	Modèle à télécharger depuis HuggingFace
-ngl	99	Charge toutes les couches sur le GPU Metal
-c	4096 / 8192	Taille du contexte en tokens
-fa on	—	Flash Attention (génération plus rapide)
-np	1	Utilisateurs simultanés (MTP : maximum 1)
--spec-type	mtp	Active le décodage Multi-Token Prediction
--spec-draft-n-max	3	Nombre de tokens prédits simultanément

5. Utilisation

Interface web (navigateur)

Une fois le serveur lancé, tu verras dans le terminal :

```
llama server listening at http://127.0.0.1:8080
```

Ouvre ton navigateur et va sur <http://localhost:8080>. Tu as une interface de chat complète. Le terminal reste occupé par le serveur — c'est normal, ne le ferme pas.

Interface terminal (llama-cli)

```
./llama.cpp/llama-cli \  
-hf unsloth/Qwen3.6-27B-MTP-GGUF:UD-Q4_K_XL \  
-ngl 99 -fa on \  
--spec-type mtp --spec-draft-n-max 3 \  
-p "Ta question ici"
```

API compatible OpenAI

Le serveur expose une API compatible OpenAI sur le port 8080 :

```
curl http://localhost:8080/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{"model": "local", "messages": [{"role": "user", "content": "Bonjour"}]}'
```